

AutoPSV-Inspired Reranking on GSM8K: A Fixed-Pool Negative Result

Xiaoci Zhang Yunhe Dong

Department of Computational Linguistics

Heidelberg University

{xiaoci.zhang, yunhe.dong}@stud.uni-heidelberg.de

Abstract

AutoPSV motivates deriving process labels from changes in an outcome verifier’s prefix scores. We study a resource-constrained variant for reranking mathematical solutions, using Qwen2.5-0.5B-Instruct with 4-bit NF4 quantization and LoRA. The released training set contains 400 trajectories from 80 GSM8K official-test questions, comprising 5,302 labeled prefixes; a separate 20-question subset is used for validation. Unlike the original AutoPSV formulation, this variant thresholds *absolute* adjacent score differences and trains its process classifier with binary cross-entropy. On 200 additional, question-disjoint GSM8K official-test problems with five generated candidates each, Min and Product aggregation attain 34.0% (68/200) and 33.5% (67/200), compared with 42.0% (84/200) for answer-frequency voting. The fixed candidate pool contains a correct answer for 119/200 problems. A paired McNemar test for Min versus voting gives $p = 0.01496$. We document parsing failures (152/1,000 candidates), selection disagreements, and two illustrative score traces. These results characterize this particular lightweight implementation and candidate pool; they neither reproduce nor refute the full AutoPSV system.

1 Introduction

Large Language Models (LLMs) have demonstrated substantial capabilities in multi-step deductive and arithmetic reasoning (Wei et al., 2022; Brown et al., 2020). Nonetheless, due to their autoregressive decoding nature, intermediate errors inevitably cascade, culminating in spurious derivations and invalid conclusions. To enhance test-time execution, sampling multiple candidate trajectories per question has become standard practice. The operational challenge consequently shifts from generation to selection: How can an evaluation agent reliably select the ground-truth trajectory among multiple plausible candidates?

A primary baseline is unsupervised Self-Consistency or Majority Voting (Wang et al., 2023b), which aggregates candidate solutions via modal final answers. Despite its simplicity and competitive empirical performance, Majority Voting suffers from systematic structural vulnerabilities: when multiple sampled trajectories share an identical invalid assumption, arithmetic error, or systematic prompt bias, modal aggregation collapses onto an incorrect consensus.

To overcome the blind spots of majority voting, verification approaches have bifurcated into Outcome-Supervised Verifiers (OSVs) (Cobbe et al., 2021) and Process-Supervised Verifiers (PSVs) (Lightman et al., 2023; Uesato et al., 2022). While OSVs only assign scalar rewards based on the terminal correctness of an entire chain, PSVs assess individual reasoning steps, pinpointing the exact emergence of logical or calculation errors. Historically, training PSVs necessitated either prohibitive human annotation campaigns (Lightman et al., 2023) or massive Monte Carlo Tree Search (MCTS) rollouts (Wang et al., 2023a), which can require upwards of 10 to 12 GPU-days for moderately sized benchmarks.

To circumvent this annotation bottleneck, Lu et al. (2024) introduced AutoPSV, demonstrating that an outcome-trained verifier’s internal confidence variations across adjacent steps (Δ_{conf}^t) can automatically generate step-level supervision targets without human intervention or future-state rollouts.

While original investigations utilized multi-billion parameter foundations (e.g., Mistral-7B, Mixtral-8x7B, Qwen-72B) trained over hundreds of thousands of full-parameter tokens on multi-GPU server clusters, real-world deployment frequently encounters stringent computational limitations. Computational linguistics research and edge deployments often demand running verification on commodity hardware, high-performance comput-

ing (HPC) short-queue debugging partitions, or small-footprint models. This raises fundamental scientific and engineering questions:

- RQ1. Feasibility and Implementation:** Can an AutoPSV-inspired confidence-difference labeling pipeline be trained with an ultra-compact language model under LoRA and 4-bit quantization?
- RQ2. Verification Superiority:** Under low-resource constraints, can a lightweight process verifier reliably outperform the standard majority voting consensus in mathematical candidate reranking?
- RQ3. Observed Failure Modes:** What do the released candidate scores, answer extraction failures, and selection disagreements reveal about this run?

To address these questions, we analyze a fixed candidate-reranking experiment on GSM8K (Cobbe et al., 2021). The released train, validation, and evaluation subsets contain 80, 20, and 200 distinct questions, respectively, with no exact question-text overlap. All three subsets were drawn from the GSM8K *official test split*; the 200 evaluation questions were held out from this experiment’s training and validation. This partition does not rule out base-model pretraining contamination. Using a 4-bit quantized Qwen2.5-0.5B-Instruct backbone, we evaluate reranking of 1,000 generated candidates.

Neither Min nor Product aggregation surpasses answer-frequency voting in this fixed experiment. We inspect paired outcomes and two step-score traces to describe plausible contributors to this gap. The available artifacts do not support a causal attribution to model size, quantization, segmentation, or sequence length in isolation.

2 Related Work

2.1 Reasoning and Test-Time Search in LLMs

Chain-of-Thought (CoT) prompting (Wei et al., 2022) significantly improves complex multi-step reasoning by explicitly instructing models to produce intermediate verbal derivations prior to the terminal answer. Subsequent efforts expanded linear CoT into deliberate tree-structured or graph-structured search spaces, such as Tree-of-Thoughts (Yao et al., 2023) and program-aided generation (Gao et al., 2023). However, greedy decoding often

yields suboptimal derivations, motivating multi-path sampling strategies (Pass@K) coupled with selection mechanisms.

2.2 Outcome vs. Process Supervision

The selection of candidate solutions has traditionally relied on outcome-based verification (OSV) (Cobbe et al., 2021; Yu et al., 2023), wherein a reward head predicts whether a complete solution trajectory S_i leads to the correct final answer $y_i \in \{0, 1\}$. Yu et al. (2023) proved that an OSV trained via mean squared error (MSE) over step prefixes mathematically approximates the conditional probability of reaching a correct solution from that intermediate state, $f_\theta(S^{(1:t)}; q) \approx p(\hat{a} \mid S^{(1:t)}, q)$.

Despite theoretical consistency, OSVs suffer from outcome bias: they reward incorrect reasoning paths that arrive at the correct terminal answer by chance (false positives), and harshly penalize reasoning paths that execute multiple flawless reasoning steps but fail due to a trivial terminal calculation slip (false negatives). To provide fine-grained localization, Process-Supervised Verifiers (PSVs) were introduced (Uesato et al., 2022; Lightman et al., 2023). Lightman et al. (2023) showed that PSVs substantially outperform OSVs across math competition problems (PRM800K). However, PRM800K demanded hundreds of hours of expert human annotation, creating severe scalability bottlenecks.

2.3 Automated Process Annotation and AutoPSV

To bypass manual labeling, model-induced process supervision emerged. Wang et al. (2023a) developed Math-Shepherd, using Monte Carlo rollouts to estimate step completion probabilities. Similarly, Wang et al. (2024) investigated automated MCTS rollouts. However, MCTS methods impose massive compute costs: rolling out 8 paths per step across 15,000 problems requires billions of generated tokens and upwards of two weeks of execution on enterprise clusters (Lu et al., 2024).

Addressing this efficiency gap, Lu et al. (2024) introduced AutoPSV. AutoPSV trains an initial outcome-supervised value head and detects relative drops in model confidence across adjacent steps (Δ_{conf}^t). If the relative confidence drop surpasses a designated threshold θ , the step is labeled incorrect, and all subsequent steps are marked invalid under the first-error assumption. AutoPSV reduced token annotation costs by over 95% com-

pared to MCTS while matching or exceeding verifier performance on Mistral-7B and Qwen-72B backbones. Our work directly examines whether this premise remains viable when scaled down to ultra-lightweight architectures.

3 Methodology

3.1 Problem Formulation

Let $q \in \mathcal{Q}$ denote a natural language mathematical problem drawn from the question distribution. An autoregressive language model acts as a candidate generator G_ϕ , producing K candidate solutions $\mathcal{S}_q = \{S_1, S_2, \dots, S_K\}$. Each candidate solution S_i comprises a sequence of m_i discrete reasoning steps, denoted as:

$$S_i = (s_i^1, s_i^2, \dots, s_i^{m_i}) \quad (1)$$

Let $S_i^{(1:t)} = (s_i^1, \dots, s_i^t)$ represent the prefix trajectory up to step t . Each candidate terminates with an extracted numerical answer $\hat{a}_i = \text{Extract}(S_i)$. The ground-truth answer is denoted by a_q^* . The candidate’s outcome correctness is binary: $y_i = \mathbb{I}(\hat{a}_i = a_q^*)$.

The objective of candidate reranking is to define a scoring function $\mathcal{R}(S_i; q)$ parameterized by a verifier such that selecting the highest-scoring candidate maximizes the probability of correctness:

$$i^* = \arg \max_{i \in \{1, \dots, K\}} \mathcal{R}(S_i; q) \quad (2)$$

3.2 Automated Process Annotation with Absolute Score Differences

AutoPSV uses changes in an outcome verifier’s prefix confidence as a source of process supervision (Lu et al., 2024). The released training rows contain outcome-verifier prefix scores $p_t \in [0, 1]$ and pseudo-labels. The implemented rule uses an *absolute* adjacent difference,

$$d_t = p_t - p_{t-1}, \quad t = 2, \dots, m_i, \quad (3)$$

not the relative ratio in the original AutoPSV formulation. With threshold $\tau = -0.50$, the first-error rule is

$$y_i^1 = 1, \quad y_i^t = \mathbb{I} \left[\min_{2 \leq j \leq t} d_j > \tau \right], \quad t \geq 2. \quad (4)$$

The first step is assigned a positive label by convention because no question-only score p_0 was supplied. This label is not an observed correctness

judgment. All 400 released training rows specify `annotation_formula=absolute`, `theta=-0.5`, and `first_step_policy=positive`; their labels agree with this rule. The repository does not include the upstream outcome-verifier checkpoint and score-generation run needed to reproduce the stored p_t values independently. Thus our analysis can verify label arithmetic, but not the provenance or calibration of those scores.

3.3 Parameter-Efficient Architecture

To assess verifier viability under extreme resource constraints, we adopt Qwen2.5-0.5B-Instruct as the base backbone. To enable execution within restricted GPU memory envelopes without out-of-memory (OOM) faults, we introduce two architectural adaptations:

- **4-bit NormalFloat Quantization (NF4):** Base model parameters $W \in \mathbb{R}^{d_1 \times d_2}$ are frozen and quantized into 4-bit representations with double quantization to minimize quantization-induced error.
- **Low-Rank Adaptation (LoRA):** Trainable rank-decomposition matrices $A \in \mathbb{R}^{r \times d_2}$ and $B \in \mathbb{R}^{d_1 \times r}$ are injected into attention projection layers (W_q, W_v):

$$W' = W + \frac{\alpha}{r} BA \quad (5)$$

where rank $r = 8$ and scaling factor $\alpha = 16$.

- **Scalar Classification Head:** The sequence-classification model produces a scalar logit z_t for each encoded question-plus-prefix input; its sigmoid $v_t = \sigma(z_t) \in [0, 1]$ is used for candidate scoring. The implemented prefix construction joins the first t segmented steps and does not score explicit end-of-step delimiter tokens.

The released run trained on 5,302 prefix examples with binary cross-entropy on logits (BCEWithLogits), not mean squared error. For a minibatch B of prefix examples, the implemented objective is

$$\mathcal{L}_B = \frac{1}{|B|} \sum_{(i,t) \in B} [\log(1 + e^{z_{it}}) - y_i^t z_{it}]. \quad (6)$$

Prefix examples are sampled as individual training items; the code does not first average loss within each trajectory. This is another departure from the trajectory-normalized objective originally described in this report.

3.4 Candidate Score Aggregation

Given predicted step probabilities $\{v_1, v_2, \dots, v_{m_i}\}$ for candidate S_i , we implement two distinct mathematical aggregation paradigms:

1. **Min Aggregation ($\mathcal{R}_{\min}$):** Guided by the weakest-link principle, the solution’s quality is bounded by its lowest-scoring intermediate transition:

$$\mathcal{R}_{\min}(S_i) = \min_{1 \leq t \leq m_i} v_t \quad (7)$$

2. **Product Aggregation ($\mathcal{R}_{\text{prod}}$):** As a heuristic that compounds step scores, we sum their logarithms. The prefix scores are correlated and are not treated as independent calibrated probabilities:

$$\mathcal{R}_{\text{prod}}(S_i) = \sum_{t=1}^{m_i} \log(\max(v_t, 10^{-30})) \quad (8)$$

The floor avoids taking the logarithm of zero.

4 Experimental Setup

4.1 HPC Infrastructure and Job Scheduling

All training, prefix extraction, and evaluation workloads were orchestrated on the bwUniCluster 3.0 supercomputing infrastructure. To adhere to strict operational guidelines:

- **Execution Environment:** Workloads were allocated on accelerated debugging partitions (`dev_gpu_4_a100`) equipped with NVIDIA A100 Tensor Core GPUs (40GB VRAM) and AMD EPYC 7742 processors.
- **Slurm Job Encapsulation:** Non-interactive tasks were submitted via automated Slurm batch scripts with strict walltime limits ($T \leq 01:00:00$) and 30GB host memory allocations.
- **Cache Isolation:** All Hugging Face and PyTorch runtime caches were redirected to non-volatile local scratch partitions (`/scratch/$USER`) to prevent home-directory disk quota saturation.

4.2 Training Protocol and Hyperparameters

The released training file contains 400 candidate trajectories for 80 distinct questions drawn from the first part of the GSM8K official test split, decomposed into 5,302 prefix examples. A separate file contains 100 trajectories for 20 additional validation questions. These files are disjoint by exact

Hyperparameter	Assigned Value
Backbone Model	Qwen2.5-0.5B-Instruct
Quantization Precision	4-bit NF4 (Double Quant)
LoRA Rank (r)	8
LoRA Alpha (α)	16
LoRA Target Modules	W_q, W_v
LoRA Dropout	0.05
Trainable Parameters	0.542M / 494.6M (0.110%)
Optimizer	AdamW
Learning Rate	2×10^{-4}
LR Schedule	Linear (0 warmup)
Weight Decay	0 (default)
Loss	BCE with logits
Batch Size (Per Device)	4
Gradient Accumulation Steps	2
Total Update Steps	30
Training Trajectories	400
Prefix Sample Count	5,302
Labeling Threshold (θ)	-0.50

Table 1: Training configurations for lightweight process verifier on the HPC accelerated debugging node.

question text from the final 200-question evaluation subset. Table 1 records the settings in the training code and run report. The per-device batch size was 4 with two accumulation steps, giving an effective batch size of 8 across 30 optimizer updates. The code uses AdamW and a linear learning-rate schedule with no configured warmup or nonzero weight decay.

4.3 Question Separation and Generation Parameters

The fixed evaluation protocol uses the following controls and settings:

- **Question Separation:** The 80 training, 20 validation, and 200 final-evaluation questions have no exact question-text overlap in the released files. All are drawn from the GSM8K official test split. The final question-file SHA-256 begins `675d924f`; a file hash identifies the frozen question set but does not by itself establish disjointness or exclude model-pretraining exposure. The selection manifest documents exclusion of local prior records and explicitly does not claim to exclude pretraining contamination.
- **Sampling Configuration:** For each problem, 5 candidate solutions were generated using the frozen Qwen2.5-0.5B-Instruct model under stochastic decoding: temperature $T = 0.7$, top- $p = 0.9$, with maximum generated tokens constrained to 768.
- **Prompt Formatting:** The generator was in-

structed to end with a line of the form #### <number>, where number is a placeholder for its numeric answer. The actual prompt also requested at most six short reasoning steps and no text after the answer line.

- **Throughput:** Generation of all 1,000 candidate chains required 3,775.73 seconds ($\approx$ 62.9 minutes) of sustained compute.

4.4 Checkpoint Persistence and Reload Audit

In machine learning deployments, non-deterministic model serialization or adapter re-initialization frequently introduces silent inference discrepancies. We executed an explicit post-training persistence audit:

$$\text{Error}_{\text{reload}} = |z^{\text{saved}}(x_{\text{probe}}) - z^{\text{reloaded}}(x_{\text{probe}})| \quad (9)$$

The stored audit compares one probe input before saving and after reloading. Its *raw logit*, not a sigmoid probability, was 1.2109375 both times, for a recorded absolute difference of 0.0. This confirms agreement on that probe in the recorded environment; it does not establish agreement over an entire audit set or across other hardware.

4.5 Answer Extraction and Penalization Protocol

Numerical answers were extracted from the final nonempty line using the repository’s fixed parser, which accepts boxed answers, answers marked with the requested four-hash delimiter, and some explicit final-line expressions. Of the 1,000 generated candidates, 152 yielded no extracted answer. Five of these 152 ended at the generation length limit; the remaining 147 have an EOS finish reason. The released data do not establish a finer causal breakdown of parsing failures.

Standardized Evaluation Rule: All 152 unextractable candidates were treated as invalid ($y = 0$). In Majority Voting, unextractable candidates were discarded from the voting pool, allowing valid candidates to establish consensus. In verifier reranking, unextractable candidates were permitted to compete based on their step scores; if selected, they were evaluated as incorrect.

Method	Correct	Acc.	95% CI
Min	68/200	34.0%	[27.8%, 40.8%]
Product	67/200	33.5%	[27.3%, 40.3%]
Voting	84/200	42.0%	[35.4%, 48.9%]
Oracle	119/200	59.5%	[52.6%, 66.1%]

Table 2: Quantitative performance across 200 GSM8K test instances (1,000 candidates). Wilson score intervals computed at $\alpha = 0.05$. Unextractable candidates (152/1000) are scored as incorrect.

5 Quantitative Results

5.1 Main Benchmark Performance

Table 2 reports the performance of the lightweight process-supervised verifier across the 200 formal test problems against baseline selection paradigms, accompanied by 95% Wilson score confidence intervals.

The empirical results document that neither process verification approach exceeds the consensus baseline:

- **Majority Voting** achieves 42.0% accuracy (84/200) using extracted terminal-answer frequencies and the fixed candidate-ID tie rule.
- **Min Aggregation** achieves 34.0% accuracy (68/200), trailing Majority Voting by 8.0 percentage points.
- **Product Aggregation** achieves 33.5% accuracy (67/200), trailing Majority Voting by 8.5 percentage points.
- **Oracle Pass@5:** The candidate pool contains at least one extractably correct answer for 119 of 200 problems (59.5%). This is a hindsight upper bound for selectors restricted to this fixed pool, 17.5 percentage points above voting. On 81 problems no selector over these candidates can return an extractably correct answer.

5.2 Statistical Hypothesis Testing

For the paired Min-versus-voting outcomes, 11 questions are correct only under Min and 27 only under voting. McNemar’s continuity-corrected statistic is $\chi^2 = 5.921$ (two-sided asymptotic $p = 0.01496$; exact binomial $p = 0.01385$). This is evidence of a difference for this fixed evaluation design, conditional on its question-selection and parsing rules. It does not isolate which implementation choice caused the gap.

Categorical Regime	Count	Ratio (%)
Majority Correct $\cap$ Min Correct	57	28.5%
Majority Wrong $\cap$ Min Wrong	105	52.5%
Positive Recovery (Maj. 0, Min 1)	11	5.5%
False Attribution (Maj. 1, Min 0)	27	13.5%
Oracle Correct $\cap$ Maj. 0 $\cap$ Min 0	24	12.0%
Oracle Wrong (Zero Valid Candidates)	81	40.5%

Table 3: Paired outcomes for Majority Voting and Min across 200 questions. The last two rows partition the 105 questions missed by both methods; they are not additional cells.

	Product = 0	Product = 1	Total
Min = 0	125	7	132
Min = 1	8	60	68
Total	133	67	200

Table 4: Confusion matrix comparing candidate correctness between Min and Product aggregation functions.

5.3 Cross-Method Discrepancy Matrix

Table 3 gives four mutually exclusive paired-outcome cells, followed by two *subsets* of the Majority-wrong/Min-wrong cell. The final two rows must not be added to the first four.

The cross-tabulation highlights two critical behavioral dynamics:

1. **Recovery (11 Cases, 5.5%)**: Voting returned an incorrect answer while Min selected a correct candidate. Voting can involve a tie resolved by candidate ID, so these are not necessarily corrections of a strict majority.
2. **Degradation (27 Cases, 13.5%)**: Voting returned a correct answer while Min selected an incorrect or unparseable candidate.

The 27 degradations exceed the 11 recoveries by 16 questions, exactly matching the accuracy difference of eight percentage points.

5.4 Min vs. Product Decision Dynamics

Table 4 investigates the decision correlation between Min and Product aggregation functions. Min and Product selected the exact same candidate in 124 out of 200 problems (62.0%). Across the entire test split, 60 problems were solved correctly by both methods, while 125 problems were missed by both. Min uniquely solved 8 problems where Product failed, and Product uniquely solved 7 problems where Min failed.

5.5 Reasoning Chain Length Bias

To assess whether aggregation functions exhibit systemic length bias, we compute the distribution of reasoning step lengths across selected candidates:

- Candidates selected by Min Aggregation possess an average step length of 11.52 ± 4.45 steps (range: [4, 26]). Candidates that were correct averaged 10.60 ± 3.92 steps, whereas candidates that were incorrect averaged 11.99 ± 4.63 steps.
- Candidates selected by Product Aggregation possess an average step length of 10.28 ± 3.77 steps (range: [3, 25]).
- Paired step-length difference ($\text{Steps}_{\text{min}} - \text{Steps}_{\text{prod}}$) shows a mean delta of +1.24 steps (median: 0.0, max: +15.0).

At fixed existing scores, adding a step with $v_t < 1$ lowers the unnormalized Product score, creating a mathematical length dependence. The selected Product candidates are shorter on average in this run. This observational comparison does not establish that length alone explains their errors or that any candidate was actually truncated during reranking.

6 Qualitative Error Taxonomy and Case Studies

The 38 paired-outcome disagreements (11 recoveries, 27 degradations) motivate inspection of the released traces. We report directly verifiable patterns and hypotheses separately; the repository does not include a per-case human-coded error taxonomy or an inter-annotator audit.

6.1 Mechanistic Audit of Contrasting Cases

Table 5 provides an in-depth trace audit of two primary representative cases drawn from our formal test logs.

Case 1: Recovery with Two Tie Breaks (gsm8k-test-118) The problem states: "A customs officer counted 2 containers of imported vehicles, each having 5 vehicles inside. The next day, more containers were brought in, and the total number of vehicles became 30. Calculate the number of containers imported on the second day."

- **Voting**: Two candidates answer 20, two answer 4, and one answers 6. Voting returns 20 because its fixed candidate-ID tie rule favors c0 ; 20 is not a

Candidate ID	Critical Step / Error Mechanism	Steps	$\mathcal{R}_{\min}$	Parsed / Gold
Case 1: Tie-Sensitive Recovery (<i>gsm8k-test-118</i> , Gold Answer: 4)				
118-c0 (voting tie winner)	Set $10 + x = 30 \Rightarrow x = 20$, conflating containers and vehicles.	9	0.3355	20.0 (Incorrect)
118-c2 (Min tie winner)	Computes $2 \times 5 = 10$, $30 - 10 = 20$, and $20/5 = 4$.	8	0.4843	4.0 (Correct)
118-c4 (Min tied candidate)	Gives the incorrect answer 20; has exactly the same minimum score as c2.	10	0.4843	20.0 (Incorrect)
Case 2: False Attribution and Length Drift (<i>gsm8k-test-1043</i> , Gold Answer: 130000)				
1043-c0 (Voting answer)	Computes $26 \times 5 \times 1000 = 130000$; its final scored step receives $v_{12} = 0.0159$.	12	0.0159	130000.0 (Correct)
1043-c2 (Min choice)	Omits the five cities per country and computes $26 \times 1000 = 26000$; $v_6 = 0.3576$.	7	0.3576	26000.0 (Incorrect)

Table 5: Selected trace observations. In Case 1, both voting (2–2–1) and Min (c2/c4) require the specified candidate-ID tie rule. Case 2 shows a low final score on one correct trajectory; it does not isolate sequence length as the cause.

strict majority. Candidate c0 conflates additional containers with vehicles in $10 + x = 30$.

- **Verifier Intervention:** The verifier assigned step scores of $[0.994, 0.949, 0.843, 0.997, 0.981]$ to the initial steps, but dropped to $v_6 = 0.3355$ upon evaluating the invalid algebraic transition.
- **Min Selection:** Candidate c2 correctly computes $(30 - 10)/5 = 4$. Its minimum step score is 0.4842581451, greater than c0’s 0.3354596794. However, incorrect candidate c4 has *exactly the same* minimum score as c2. The fixed candidate-ID tie rule selects c2. This is a tie-sensitive recovery, not unambiguous evidence of correct error localization.

Case 2: Low Terminal Score on a Correct Candidate (*gsm8k-test-1043*) The problem states: “*There are 26 countries in South America and, in each country, there are 5 cities with 1000 people living in each city... how many people are there in South America?*”

- **Correct Candidate (c0):** Candidate c0 computes $26 \times 5 \times 1000 = 130,000$. Its first nine prefix scores exceed 0.72, while its twelfth and final score is $v_{12} = 0.0159$. The trace shows a terminal score drop but does not identify its cause.
- **Incorrect Candidate (c2):** Candidate c2 omits the factor of five and computes $26 \times 1000 = 26,000$. Its seven scored steps have minimum $v_6 = 0.3576$.
- **Pathological Inversion:** Min aggregation compared the candidates via their minimum scores:

$$\mathcal{R}_{\min}(c2) = 0.3576 > 0.0159 = \mathcal{R}_{\min}(c0)$$

Min therefore selects the incorrect c2 over correct c0. Length, final-answer formatting, and segmentation remain possible explanations rather than isolated causes.

6.2 Observed Patterns and Untested Explanations

The released artifacts establish several facts. Among the 27 voting-correct/Min-wrong questions, Min selects an unparseable candidate on five. Across all 200 questions, Min selects 29 unparseable candidates. Of the 1,000 generated candidates, 152 are unparseable, and only six candidates in total have a length-limit finish reason. The 12-step correct candidate in Case 2 receives a very low terminal score, while a seven-step incorrect candidate receives a higher minimum. These are concrete contributors to particular errors under the fixed evaluation rule.

Other proposed mechanisms require additional tests. An intervention that varies segmentation while holding candidate content fixed would test segmentation sensitivity; a length-controlled comparison would test position effects; and matched models or training runs would be needed to attribute the result to parameter count, NF4 quantization, or training duration. The four-category percentages in the earlier draft cannot be reconstructed from a released per-case annotation file and are therefore omitted.

7 Discussion and Critical Reflections

7.1 How Far Can This Result Generalize?

The original AutoPSV study reports gains with much larger models and a different annotation and

training configuration (Lu et al., 2024). This experiment changes model scale, optimization budget, score-difference formula, and loss together. Its negative result cannot establish a lower bound on viable model scale or isolate why the original system performs differently.

Representation and Scoring Case 2 is consistent with a verifier failing to track the five-cities-per-country condition, but it does not prove that model capacity is the cause. The same candidate pool could be scored with alternative model sizes, training budgets, or aggregation rules in a future ablation.

The Sample and Optimization Regimes The released run uses 400 training trajectories, 5,302 prefix examples, and 30 gradient updates. The code and run report verify these counts. No held-out probability-calibration analysis or training-duration ablation is released, so the adequacy of 30 steps and calibration of v_t remain open questions. The upstream outcome verifier that produced training-prefix scores is also absent from this repository.

7.2 Implications for Candidate Reranking Systems

Our negative results carry concrete engineering implications:

- **Voting Baseline:** Under this candidate-generation and extraction protocol, voting exceeds Min by 8.0 and Product by 8.5 percentage points. This comparison should be reported alongside the tie-break rule.
- **Length and Aggregation:** Product’s sum of log scores is length-dependent by construction. Length-normalized scoring is a plausible future ablation, not a tested improvement in this report.
- **Parsing:** Restricting verifier selection to parseable candidates is another possible ablation. It was not part of the frozen final evaluation and should not be reported as an achieved result.

8 Conclusion

We evaluated an AutoPSV-inspired, 4-bit LoRA Qwen2.5-0.5B process classifier for reranking five candidates on each of 200 question-disjoint GSM8K instances. This is a lightweight variant rather than a faithful replication of the original AutoPSV method.

The fixed results show 11 questions corrected by Min relative to voting and 27 degraded, for a net loss of 16 solved questions. Product is also below voting. The released traces expose individual score inversions and the effect of unparseable selections, but they do not establish general scaling boundaries or unique causal mechanisms. The value of this result is a transparent benchmark and a set of testable follow-up hypotheses.

Limitations

The scope of the evidence is limited in several ways:

1. **Method Variation:** Absolute score differences, a conventionally positive first step, BCE training, quantization, LoRA, and a small update budget differ from the original AutoPSV setup. The present results neither replicate nor invalidate the original paper (Lu et al., 2024).
2. **Data and Provenance:** Training and validation questions come from the official GSM8K test split, although they are question-disjoint from the 200 final evaluation items. The available screening record excludes local prior questions, not possible base-model pretraining exposure. The upstream score-generation code/checkpoint and a per-case qualitative audit record are not included.
3. **No Causal Ablations:** The effects of quantization, model size, training budget, segmentation, answer filtering, and score aggregation cannot be separated by this run. Evaluation is limited to GSM8K, one generator, one verifier checkpoint, and a 200-question sample.

Ethical Considerations

This study utilizes public, open-source mathematical benchmark datasets (GSM8K) and open-weights foundation models (Qwen2.5). The research poses no risks regarding personally identifiable information, toxic text generation, or dual-use societal harms. All compute was hosted on certified academic infrastructure.

References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot

learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International Conference on Machine Learning (ICML)*, pages 10764–10799. PMLR.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*.

Jianqiao Lu, Zhiyang Dou, Hongru Wang, Zeyu Cao, Jianbo Dai, Yingjia Wan, and Zhijiang Guo. 2024. AutoPSV: Automated process-supervised verifier. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Y Wu, and Zhifang Sui. 2023a. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR)*.

Zihan Wang, Yunxuan Li, Yuexin Wu, Liangchen Luo, Le Hou, Hongkun Yu, and Jingbo Shang. 2024. Multi-step problem solving through a verifier: An empirical analysis on model-induced process supervision. *arXiv preprint arXiv:2402.02658*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems (NeurIPS)*.

Fei Yu, Anningzhe Gao, and Benyou Wang. 2023. Outcome-supervised verifiers for planning in mathematical reasoning. *arXiv preprint arXiv:2311.09724*.

A Step-Level Case Traces

Tables 6 and 7 summarize the final process classifier’s sigmoid scores. Text previews are abbreviated; complete candidate text and scores are in the released repository. The values below are *not* the upstream outcome-verifier scores used to make training labels.

Step	Text preview	v_t
118-c0: answer 20, Min 0.3355		
1	Two containers, five vehicles each	0.9944
2	$2 \times 5 = 10$ vehicles	0.9489
3	Day-two total is 30 vehicles	0.8429
4	Let x count added containers	0.9974
5	Total after adding x containers	0.9809
6	$10 + x = 30$	0.3355
7	$x = 30 - 10$	0.8289
8	$x = 20$	0.6715
9	Final answer 20 containers	0.5361
118-c2: answer 4, Min 0.4843		
1	Two containers, five vehicles each	0.9924
2	$2 \times 5 = 10$ vehicles	0.9408
3	Total is now 30 vehicles	0.8670
4	Find $30 - 10$ additional vehicles	0.9305
5	Difference is 20 vehicles	0.8529
6	Divide 20 by five	0.9344
7	$20/5 = 4$ containers	0.4843
8	Final answer 4	0.7402

Table 6: Abbreviated step trace for Case 118. Incorrect candidate c4, omitted for space, ties c2 at the exact Min score 0.4842581451; candidate-ID order selects c2.

Step	Text preview	v_t
1043-c0: answer 130000, Min 0.0159		
1	Count 26 countries	0.9161
2	Five cities in each country	0.7387
3	One city has 1000 people	0.9890
4	Population of each city	0.7210
5	Cities within one country	0.8141
6	1000 people, five cities	0.7280
7	$1000 \times 5 = 5000$ per country	0.9489
8	Total in South America	0.9305
9	$26 \times 5000 = 130000$	0.9073
10	Summary context	0.5250
11	Summary wrap-up	0.3549
12	Final answer 130000	0.0159
1043-c2: answer 26000, Min 0.3576		
1	Count 26 countries	0.6361
2	Incorrectly set cities to 26	0.8278
3	1000 people per city	0.9579
4	Population per city	0.6132
5	Multiply cities and people	0.6783
6	$26 \times 1000 = 26000$	0.3576
7	Final answer 26000	0.9887

Table 7: Abbreviated step trace for Case 1043. The terminal score drop on correct c0 is observed; this trace does not isolate its cause.